# INHERITANCE

**Objective** • Concept of Inheritance

## INTRODUCTION

-*C++* strongly supports the concept of *reusability.* The C++ classes reused in several ways. Once a class has been written and tested, it can be adapted by other programmers to suit their requirements. This is basically done by creating new reusing the properties of the existing ones

***The mechanism of deriving a new class from an old one is called inheritance (or derivation). The old class is referred to as the base class and the new one is called the derived class or subclass.***

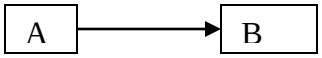
**Types/Forms of Inheritance**

***"What do you recollect when you talk about inheritance? Doesn't it immediately relate to imbibing/inheriting certain characteristics from our parents? Absolutely we are talking about 'taking over' certain characteristics from the 'parent body' to the 'child body'. Lets explore this phenomenon of 'Inheritance' in C++ now…."***

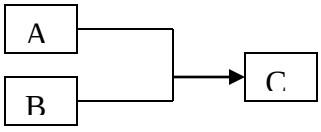
--Reusability is yet another important feature of OOP. It is always nice if we could reuse something that already exists rather than trying to create the same all over again. ***It, saves time and money but also increase reliability.*** For instance the reuse of a class that has already been tested, debugged and used many times can the effort of developing and testing the same again.

-The derived class inherits some or all of the traits from the base class. A class can also inherit properties from more than one class or from more than one level. A derived class with one base class, is called ***single inheritance*** and one with several base classes is called ***multiple inheritance***. On the other hand, the traits of one class may be inherited by more than one class . This process is known as ***hierarchical inheritance***. The mechanism of deriving from another 'derived class' is known as ***multilevel inheritance***. *Fig* . shows forms of inheritance that could be used for writing extensible
programs. The direction , indicates the direction of inheritance. (Some authors show the arrow in opposite

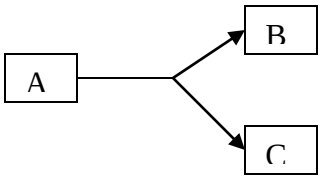
(a) single inheritance



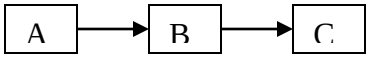

(b) multiple inheritance



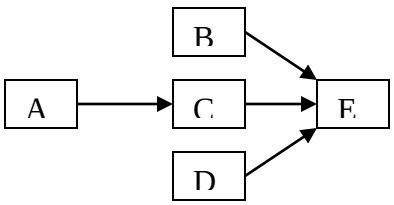

(c)Hirarchical inheritance




(d)Multilevel inheritance




(e)Hybrid inheritance



**DEFINING DERIVED CLASSES**

Derived class can be defined by specifying its relationship with the base class in addition own details. The general form of defining a derived class is:

```
c1ass  derived_class_name : visibility_mode   base_c1ass_name
{
……//
……// members of derived class
……//
};
```

The colon indicates that the *derived-class-name* is derived from the *base-class-nam*, *visibility-mode* is optional and, if present, may be either private or public. The default visibility, mode is private. Visibility mode specifies whether the features of the base class are *privately derived or publicly derived.*

Examples:

```
class ABC: private XYZ // private derivation
{
members of ABC
} ;
class ABC: public XYZ // public derivation
{
members of ABC
} ;
class ABC: XYZ // private derivation by default
{
members of ABC
};
```

When a base class is ***privately inherited*** **by a derived class**, 'public members' of the class become 'private members' of the derived class and therefore the public members of base class can only be accessed by the member functions of the derived class. They are inaccessible to the objects of the derived class. Remember, a public member of a class can accessed by its own objects using the *dot operator*. The result is that no member of the class is accessible to the objects of the derived class.

When a base class is ***publicly inherited*** **by a derived class**, 'public members' of the class become 'public members' of the derived class and therefore the public members of base class can be accessed by the objects of the derived class.

**SINGLE INHERITANCE**

Let us consider a simple example to illustrate inheritance. Program shows a base class B and derived class D. The class B contains one private data member, one public data

member and three public member functions. The class D contains one private data member and two public member functions.

```
#include <iostream>
using namespace std;
class B
{
        int a; //private - not inheritable
public:
        int b; //public, ready for inheritance
        void get_ab(void);
        int get_a(void);
        void show_a(void);
};

class D : public B // public derivation
{
        int c;
public :
        void mul(void);
        void display(void);
};


void B :: get_ab(void)
{
        a=5;b=10;
}

int B:: get_a(void)
{
        return a;
}
void B :: show_a(void)
{
        cout <<"a=" <<a<<"\n";
}
void D :: mul(void)
{
        c=b*get_a();
}
void D :: display()
{
        cout <<"a="<<get_a( )<<"\n";
        cout <<"b="<<b <<"\n";
        cout <<"c="<<c <<"\n \n ";
}
```

```
int main()
{
        D d; //object created
        d.get_ab();
        d.mul();
        d.show_a();
        d.display();
        d.b=20;
        d.mul();
        d.display();

        return 0;
}
```

Given below is the output of above Program :

```
a = 5
a = 5
b = 10
c = 50

a = 5
b = 20
c = 100
```

The class D is a public derivation of the base class B. Therefore, D inherits all the public members of B and retains their visibility. Thus a public member of the base class B is also a public member of the derived class D. The private members of B cannot be inherited by D.

## MULTILEVEL INHERITANCE

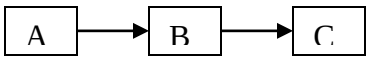
It is not uncommon that a class is derived from another derived class as shown in Fig. The class **A** serves as a base class for the derived class **B,** which in turn serves as a base class for the derived class **C**. The class **B** is known as *intermediate* base class since it provides a link for the inheritance between **A** and **C**. The chain **ABC** is known as *inheritance path.*

A derived class with multilevel inheritance is declared as follows:

class A{ }; // *Base class*
class B: public A { }; // *B derived from A*
class C: public B { }; // C *derived from B*

This process can be extended to any number of levels.

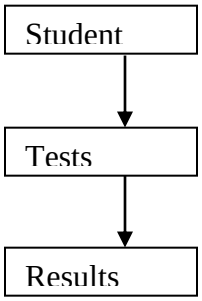
**Multilevel inheritance**




Base Intermediate base Derived

***Example***

Let us consider a simple example. Assume that the test results of a batch of students are stored in three different classes. Class **student** stores the roll-number, class **test** stores the marks obtained in two subjects and class result contains the total marks obtained in the test. **The class result can inherit the details of the marks obtained in the test and the roll number of students through multilevel inheritance i.e. .**




**Program:**

```
# include <iostream>

using namespace std;

class student
{
protected:
int roll_number;
public:
void get_number(int a);
void put_number(void);

};
void student :: get_number(int a)
{
roll_number = a;
}
void student:: put_number(void)
{
        cout << “Roll number: “ << roll_number << “\n”;
}

class test: public student // First level derivation
{
protected:
float sub1;
float sub2;
public:
void get_marks(float x, float y );
void put_marks(void);
};
void test :: get_marks(float x, float y)
{
sub1 = x;
sub2 = y;
}
void test :: put_marks(void)
{
cout << “Marks in SUB1 = “ << sub1 << “\n”;
cout << “Marks in SUB2 = “ << sub2 << “\n”;
}

class result : public test // Second level derivation

{
```

```
float total; // private by default
public:
void display(void);
};

void result :: display(void)
{
total = sub1 + sub2;
put_number();
put_marks();
cout << "Total = " << total << "\n";
}

int main()
{
result student1; //  object student1 created
student1.get_number(111);
student1.get_marks(75.0, 59.5);
student1.display();
return 0;
}
```

*Program displays the following output*:

```
Roll Number: 111
Marks in SUB1 = 75
Marks in SUB2 = 59.5
Total = 13
```

**Example2**

```
// inheritance.cpp

#include <iostream>

using namespace std;

class base //single base class

{    protected:

        int x;

        public:
```

```
        void getdata()
        {
        cout << "Enter value of x= ";
        cin >> x;
        }
};
class derive1: public base // derived class from base class
{    protected:
       int y;
        public:
        void readdata()
        {
            cout << "\nEnter value of y= "; cin >> y;
        }
};
class derive2: public derive1   // derived from class derive1
{
        private:
        int z;
        public:
```

```
        void indata()

        {

        cout << "\nEnter value of z= "; cin >> z;

        }

        void product()

        {

            cout << "\nProduct= " << x * y * z;

        }

};

int main()

{

     derive2 a;      //object of derived class

     a.getdata();

     a.readdata();

     a.indata();

     a.product();

     return 0;

}
```

**MULTIPLE INHERITANCE**

A class can inherit the attributes of two or more classes as shown in Fig.This is known as multiple inheritance. Multiple inheritance allows us to combine the features of several

existing classes as a starting point for defining new classes. It is like a child inheriting the physical features of one parent and the intelligence of another.

The syntax of a derived class with multiple base classes is as follows:

```
Class D : visibility B-1, visibility B-2…….
 { …… ……
   ……..//(body of D)
 };
```

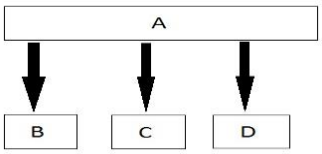
where, visibility may be either public or private. The base classes are separated by commas.
Example:

```
#include <iostream>
using namespace std;
//Classes M and N have been specified as follows:
class M
{
protected:
int m;
        public:
void get_m(int x);
};

void M :: get_m(int x)
{
        m = x;
}

class N
{
protected:
int n;
        public:
void get_n(int y);
};
void N :: get_n(int y)
{
        n = y;
}

class P: public M, public N
{

public:
void display(void);
```

```
};
// function display () can be defined as follows:
void P:: display(void)
{
        cout << "m = " << m <<"\n";
        cout << "n = " << n << "\n";
        cout << "m*n =" << m*n << "\n";
};
//The main() function which provides the user-interface may be written as follows:
int main ()
{
        P p;
        p.get_m(l0) ;
        p.get_n(20) ;
        p.display ( ) ;
        return 0;
}
```

## Hierarchical inheritance

In this type of inheritance, multiple derived classes inherit from a single base class.




This can be coded as;

```
Class A
{//data members
  //member functions
};

Class B: public A
{//data members
  //member functions
};

Class C: public A
{//data members
  //member functions
};

Class C: public D
{//data members
  //member functions
```

};

**EXAMPLE**

Using the concept of hierarchical inheritance, write a program that defines a base class named shape with a member function that gives value to its two data members, width and height/length. Then define two sub-classes triangle and rectangle, each having a member function, area *()*, to calculate the area of the shape. In the main, define two objects for a triangle and a rectangle and then call the area () function in this two objects to compute area for each shape.

```
#include <iostream>

using namespace std;

class Shape {

    //protected member variables are only accessible within
the class and its descendent classes

    protected:

        float width, height;

 //public members are accessible everywhere

    public:

 void setDimensions(float w, float h)

{

 width = w;

 height = h;

 }

};

//Class Rectangle inherites the Shape class

class Rectangle: public Shape {

    public: float area(void)

{

return (width * height);

 }

};
```

```
//Class Triangle inherits the Shape class

class Triangle: public Shape {

    public: float area(void) {

        return (width * height / 2);

}

};

//Defining the main method to access the members of the
class

int main() {

//Declaring the Class objects to access the class members

    Rectangle rectangle;

    Triangle triangle;

    rectangle.setDimensions(5, 3);

    triangle.setDimensions(2, 5);

cout << "\nArea of the Rectangle computed using Rectangle
Class is : " << rectangle.area() << "\n";

cout << "Area of the Triangle computed using Triangle Class
is: " << triangle.area();

return 0;

}
```